OpenShift Virtualization Components

Introduction

This service is designed to accelerate OpenShift adoption for organizations seeking to modernize their application infrastructure with container orchestration.

This engagement by a HawkStack Technologies Pvt Ltd Red Hat Certified OpenShift Architects with expertise and experience establishes a production-ready, best-practices Red Hat OpenShift environment designed for deployment in the private, public cloud, Virtualization or bare-metal.

Cloud services editions

- Red Hat OpenShift Service on AWS
- Microsoft Azure Red Hat OpenShift
- Red Hat OpenShift Dedicated
- Red Hat OpenShift on IBM Cloud

Self-managed editions

- Red Hat OpenShift Platform Plus
- Red Hat OpenShift Container Platform
- Red Hat OpenShift Kubernetes Engine
- Red Hat OpenShift Virtualization Engine

Services & add-ons

- Red Hat OpenShift AI
- Red Hat OpenShift Lightspeed
- Red Hat Virtualization
- Red Hat Quay
- Red Hat Advanced Cluster Management for Kubernetes
- Red Hat Advanced Cluster Security for Kubernetes
- Red Hat OpenShift API Management
- Red Hat OpenShift

Red Hat

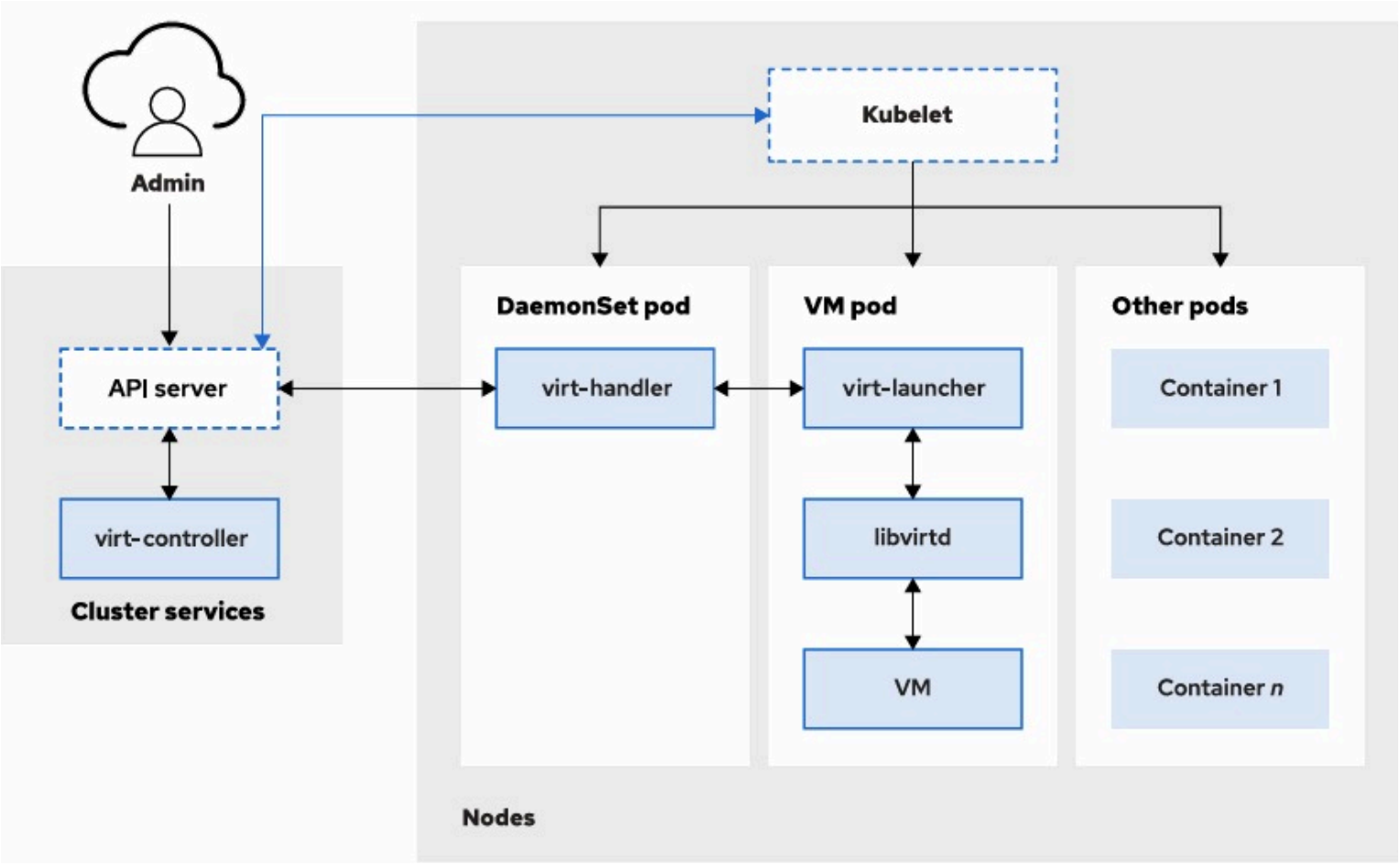
OpenShift Virtualization Components

By HawkStack Technologies Pvt Ltd

Led by Red Hat Certified OpenShift Architects

OpenShift Virtualization Components

OpenShift Virtualization supports managing and orchestrating VMs with similar tools and methods as for container management. With these shared tools and methods, administrators and developers can centralize infrastructure management.



Component	Role
Admin	The user or admin who deploys or manages the VMs.
API server	The Kubernetes API server that receives commands (create VM, delete VM, etc.).
virt-controller	A controller that watches for VM-related resources and coordinates actions.

virt-handler (DaemonSet pod)	Runs on each node, manages KVM and libvirt for VM lifecycle on that node.
virt-launcher (VM pod)	Each VM runs inside its own pod via virt-launcher.
libvirtd	Manages VM processes on the node.
VM	The actual virtual machine instance.
Kubelet	The agent on each node that ensures containers and pods run correctly.
Other pods	Regular Kubernetes containers run alongside VMs.

Example: How does it work?

Imagine you want to create a VM named test-vm to run a legacy application.

1. **Admin -> API Server:** You (Admin) create a VirtualMachine resource using oc or the OpenShift Web Console.

2. **API Server -> virt-controller:** The request hits the Kubernetes API server. The virt-controller notices the new VirtualMachine resource and decides how to launch it.

3. **virt-controller -> virt-handler:** The virt-controller instructs the virt-handler running on the selected node to handle the VM.

4. **virt-handler -> Kubelet:** The virt-handler communicates with the Kubelet to start a pod (virt-launcher) that will host the VM.

5. **virt-launcher pod -> libvirtd -> VM:** Inside the virt-launcher pod, the libvirtd process starts and manages the VM using KVM (Kernel Virtual Machine). This is how your VM boots up inside a pod, running alongside other regular containers on the node.

6. **Other Pods:** At the same time, the node might run other container pods for applications like web servers or databases — VMs and containers co-exist!

Key Point

Benefit: You can manage VMs just like containers — schedule, monitor, and orchestrate them with Kubernetes-native tools

So your VM test-vm is now a pod, visible with oc get pods, and manageable with Kubernetes networking, storage, and security features.

Summary Example:

You: oc create -f test-vm.yaml

-> API server: Accepts the request.

-> virt-controller: Decides where to run the VM.

-> virt-handler + Kubelet: Start virt-launcher pod.

-> virt-launcher: Runs libvirtd to boot the VM. -> VM: Your legacy app is now running inside a VM inside a pod

OpenShift Virtualization Operator Components

The following components of the OpenShift Virtualization operator are described in the image:

virt-api

A cluster-level component that provides an HTTP RESTful entry point for managing VM and VMI-related workflows within the cluster. It updates virtualization Custom Resource Definitions (CRD) and handles defaulting and validating Virtual Machine Instance (VMI) CRDs.

virt-controller

This is a cluster-level component that handles cluster-wide virtualization functions and manages the lifecycle of pods associated with VMIs. The operator uses this component to create the pod where the VM objects execute.

Red Hat

HAWKSTACK

 virt-handler	A host-level DaemonSet resource that runs on each node. It monitors for changes to a VM object and performs the necessary operations to meet the desired state.
 virt-launcher	The primary container in a VMI-associated pod. The virt-launcher component provides control groups (cgroups) and namespaces for hosting the VMI process. The virt-handler passes the VM CRD object to the virt-launcher component.
 libvirtd	Every VMI pod contains a libvirtd instance for the virt-launcher component to manage the lifecycle of a VMI process.

Components Explained

virt-api

- This is the REST API entry point for virtualization in OpenShift.
- When you create a VM or VMI, your request hits this API.
- It updates the Custom Resource Definitions (CRDs) for Virtual Machines and VMIs.
- Example: When you run oc apply -f test-vm.yaml, your YAML is processed through virt-api. It stores your VM as a Kubernetes CRD.

virt-controller

- This watches the cluster for VM/VMI resources.
- It decides when and where to run the VM.
- It creates the pod to host the VM process.
- Example: Once your test-vm CRD is created, virt-controller schedules it on a node with available resources.

Red Hat

HAWKSTACK

virt-handler

- Runs on every node (DaemonSet).
- It talks to the node's Kubelet and virt-launcher.
- It ensures the VM's pod state matches the desired state.

Here is the text from the image, without emojis:

Example: Your test-vm runs inside its own virt-launcher pod. This is how the VM gets its dedicated compute resources, networking, and isolation.

libvirtd

- A hypervisor manager inside the virt-launcher pod.
- It uses KVM/QEMU under the hood to boot the VM.
- It keeps the VM running until you stop or delete it.

Example: libvirtd inside virt-launcher boots your Cirros image, assigns virtual disks, CPU, and memory — just like it would on bare metal.

Putting it all together with a flow [Flow: Create a VM (test-vm)]

1. oc apply -f -> Request hits virt-api

2. virt-controller sees a new VM CRD -> decides where to run

3. virt-handler on that node -> creates the virt-launcher pod.

4. virt-launcher -> starts libvirtd inside the pod.

5. libvirtd boots the VM -> VM runs as a normal Linux process inside the pod's container runtime.

6.When you stop the VM, virt-launcher shuts it down and the pod stops.

Why is this powerful?

- You manage VMs with the same Kubernetes tools as containers.
- VMs get full Kubernetes scheduling, networking, and storage.
- You can run containers and VMs side by side — useful for legacy apps and modern workloads together!

The HyperConverged Operator

A required component of OpenShift Virtualization is the HyperConverged Operator (HCO), which includes the following resources

Resource name	Description
`deployment/hco-webhook`	Validates the HyperConverged custom resource contents.
`deployment/hyperconverged-cluster-cli-download`	Provides the virtctl tool binaries to the cluster so that you can download them directly from the cluster.
`kubevirt/kubevirt-kubevirt-hyperconverged`	Contains all needed operators, CRs, and objects for OpenShift Virtualization.
`ssp/ssp-kubevirt-hyperconverged`	A Scheduling, Scale, and Performance (SSP) CR.
`cdi/cdi-kubevirt-hyperconverged`	A Containerized Data Importer (CDI) CR.
`networkaddonsconfig/cluster`	A CR that instructs and is managed by the Cluster Network Addons operator.

What is the HyperConverged Operator (HCO)?

The HyperConverged Operator (HCO) is a special operator in OpenShift Virtualization that combines all the necessary operators and custom resources (CRs) into a single, unified deployment. Instead of manually installing multiple components, you install the HCO, and it automatically configures everything needed to run VMs within Kubernetes. Think of the HCO as the "master operator" for your virtualization environment.

Red Hat

HAWKSTACK

Resource name	Purpose (explained)
`deployment/hco-webhook`	Validates the HyperConverged custom resource contents.
`deployment/hyperconverged-cluster-cli-download`	Provides the virtctl tool binaries to the cluster so that you can download them directly from the cluster.
`kubevirt/kubevirt-kubevirt-hyperconverged`	Contains all needed operators, CRs, and objects for OpenShift Virtualization.
`ssp/ssp-kubevirt-hyperconverged`	A Scheduling, Scale, and Performance (SSP) CR.
`cdi/cdi-kubevirt-hyperconverged`	A Containerized Data Importer (CDI) CR.
`networkaddonsconfig/cluster`	A CR that instructs and is managed by the Cluster Network Addons operator.

Example: Putting it all together

Let's say you want to create a RHEL VM from an ISO image with advanced networking.

1. HCO: You install the HCO — it deploys all these CRs for you.
2. CDI: You upload your RHEL ISO using CDI.

```
oc create -f pvc-rhel-iso.yaml
```

3. SSP: You use SSP-provided templates for vCPU/memory tuning.
4. NetworkAddonsConfig: You configure Multus or SR-IOV for VM NICs.
5. virtctl: You download virtctl from hyperconverged-cluster-cli-download:

```
oc get virtclusterversion
virtctl console my-rhel-vm
```

6. hco-webhook: Validates your VM spec on creation to ensure no mistakes.

Result

Your RHEL VM boots with the right resources, networking, storage — fully managed by the unified HCO stack.